

Pay with Quai — Deployment & Test-Run Runbook

An operational guide for standing up the Pay with Quai stack on Quai testnet (Orchard) and running the full payment loop end to end. Written directly from the source code (`contracts/` , `backend/`), including every environment variable, script, and endpoint.

The stack has two deployable parts:

Part	Directory	What it is
Contracts	<code>contracts/</code>	Hardhat project. Deploys the <code>PayWithQuai</code> UUPS proxy (the payment router) plus a testnet mock stablecoin and optional governance timelock.
Backend (relay)	<code>backend/</code>	TypeScript service: indexes <code>PaymentReceived</code> events, confirms finality, and delivers signed webhooks to merchants. Also hosts the merchant/admin HTTP API.

1. Prerequisites

- Node.js **>= 20** (the backend declares `engines.node >= 20`).
- npm (or your favorite package manager).
- A **Quai testnet wallet** whose address starts with `0x00` (Cyprus-1 zone) funded with testnet QUAIs for gas (Orchard faucet). The deployer key **MUST** be in the same zone you deploy to — Quai value cannot move across zones.
- Network access to the Orchard RPC (`https://orchard.rpc.quai.network`).
- Google Chrome not needed — this runbook's only tools are npm scripts and `curl`.

All commands below run from the project root `<repo>/contracts` and `<repo>/backend` respectively.

2. Phase 1 — Deploy the contracts (testnet)

2.1 Install and verify

```
cd contracts
npm install
npm run compile      # hardhat compile
npm test             # in-process EVM tests – no network needed, ~40s mocha timeout
```

Expected: compile succeeds (Solidity 0.8.20, london EVM — no `PUSH0`), all `test/PayWithQuai.test.js` cases pass.

2.2 Configure the deployer environment

```
cp .env.example .env
```

Variable	Required	Meaning
<code>RPC_URL</code>	yes	<code>https://orchard.rpc.quai.network</code> (default). Mainnet: <code>https://rpc.quai.network</code>
<code>CHAIN_ID</code>	yes	<code>15000</code> Orchard testnet (default). Mainnet: <code>9</code>
<code>CYPRUS1_PK</code>	yes	Deployer private key; address must start <code>0x00</code> and hold QUA I for gas
<code>FEE_RECIPIENT</code>	no	Where platform fees go (defaults to deployer). Same zone; must accept native value
<code>FEE_BPS</code>	no	Platform fee in basis points: <code>0</code> = none, <code>50</code> = 0.5%, max <code>500</code> = 5%
<code>STABLECOIN_ADDR</code>	no	Real stablecoin to allowlist (mainnet). Testnet auto-allowlists the mock
<code>MULTISIG_ADDR</code>	no	If set, deploys a <code>TimelockController</code> and hands proxy ownership to it (§2.4)
<code>TIMELOCK_MIN_DELAY</code>	no	Timelock delay, seconds (default <code>172800</code> = 48h)
<code>MERCHANT_ADDR</code>	no	<code>payDemo.js</code> only: payout wallet for the demo (defaults to deployer)

2.3 Deploy

```
npm run deploy:testnet      # == hardhat run scripts/deploy.js --network cyprus1
```

What the script deploys, in order (`contracts/scripts/deploy.js`):

1. `MockStablecoin` (mUSDQ, 6 decimals, open mint faucet) — **testnet only**; skipped when `CHAIN_ID == 9` (mainnet).
2. `PayWithQuai` implementation — the UUPS logic contract (stateless).
3. `ERC1967Proxy` — the stateful proxy, initialized with `initialize(feeRecipient, feeBps, deployer)`. **This is the address everything interacts with.**
4. The owner then allowlists **native QUA I** (`address(0)`) and the mock stablecoin (plus `STABLECOIN_ADDR` if set) via `setTokenAccepted`.

The script writes `deployments/<network>.json` (e.g. `deployments/cyprus1.json`):

```

{
  "network":      "cyprus1",
  "chainId":      15000,
  "payWithQuai":  "0x...", // <-- PROXY: use THIS address for relayer + all integrat
  "payWithQuaiImpl": "0x...",
  "timelock":      null,
  "mockStablecoin": "0x...",
  "feeRecipient":  "0x...",
  "feeBps":        "0",
  "deployer":      "0x..."
}

```

2.4 (Production) Governance hand-off

Only when `MULTISIG_ADDR` is set: the script deploys the timelock and calls `transferOwnership(timelock)`. Because the contract is `Ownable2Step`, the timelock must call `acceptOwnership()` to finish — the script prints the exact target/data to schedule through the multisig. Until then the deployer EOA remains owner. On testnet, leave `MULTISIG_ADDR` unset for fast iteration.

3. Phase 2 — Run the backend relayer

3.1 Install and verify

```

cd backend
npm install
npm run typecheck      # tsc --noEmit
npm test                # vitest: api, indexer, dispatcher, signer, store, backoff, urlGuar

```

3.2 Configure

```
cp .env.example .env
```

Variable	Default	Meaning
<code>RPC_URL</code>	— (required)	Quai zone RPC — must be the same zone as the deployed proxy (Orchard, <code>https://orchard.rpc.quai.network</code>)
<code>CHAIN_ID</code>	— (required)	Must match <code>deployments/<network>.json</code> <code>chainId</code> (<code>15000</code> testnet)
<code>PAYWITHQUAI_ADDRESS</code>	— (required)	The proxy address from the deployment record (<code>payWithQuai</code> field)
<code>START_BLOCK</code>	empty	Block to start indexing from. Set to the proxy's deployment block to capture

Variable	Default	Meaning
		history from launch; empty starts from <code>head - CONFIRMATIONS</code> on first boot
<code>CONFIRMATIONS</code>	<code>12</code>	Blocks before a <code>PaymentReceived</code> is treated as final (reorg protection)
<code>POLL_INTERVAL_MS</code>	<code>5000</code>	Indexer poll interval
<code>MAX_BLOCK_RANGE</code>	<code>2000</code>	Max blocks per <code>getLogs</code> call (chunking)
<code>WEBHOOK_MAX_ATTEMPTS</code>	<code>10</code>	Delivery attempts before permanent <code>failed</code>
<code>WEBHOOK_BASE_BACKOFF_MS</code>	<code>5000</code>	Exponential backoff base
<code>WEBHOOK_MAX_BACKOFF_MS</code>	<code>3600000</code>	Backoff cap (1h)
<code>WEBHOOK_TIMEOUT_MS</code>	<code>10000</code>	Per-delivery POST timeout
<code>WEBHOOK_ALLOW_INSECURE_URLS</code>	<code>false</code>	Set <code>true</code> only for local dev (<code>http://localhost</code> webhook targets)
<code>PORT</code>	<code>8080</code>	HTTP API port
<code>ADMIN_API_KEY</code>	— (required, ≥ 16 chars)	Bearer token for admin endpoints; generate a long random value
<code>PUBLIC_RATE_LIMIT_WINDOW_MS</code>	<code>60000</code>	Rate-limit window for the public orders route
<code>PUBLIC_RATE_LIMIT_MAX</code>	<code>60</code>	Max requests per IP per window (429 + <code>Retry-After</code> on excess)
<code>DATABASE_PATH</code>	<code>./data/relayer.db</code>	JSON state file (cursors, merchants, deliveries). Secrets are stored plaintext here — restrict access (file is created <code>0600</code> , dir <code>0700</code>)
<code>LOG_LEVEL</code>	<code>info</code>	<code>`trace</code>
<code>LOG_PRETTY</code>	<code>false</code>	Human-readable logs for local dev

3.3 Run

```

npm run dev          # tsx watch – hot reload, for development
# or production build:
npm run build        # tsc → dist/
npm start            # node dist/index.js

```

On boot you should see: `starting Pay with Quai relayer` with the chain id, contract address and confirmations, the store loading, and `HTTP API listening` on the port.

3.4 Health check

```
curl http://localhost:8080/health
```

```
{
  "status": "ok",
  "contract": "0x...",
  "chainId": 15000,
  "cursor": 1234567
}
```

- `cursor` is the last fully-indexed block — it must **advance** over time (check again after a few polls).
- Missing `cursor` right after first boot is fine if `START_BLOCK` was empty; the indexer seeds it to `head - CONFIRMATIONS`.

4. Phase 3 — Run the end-to-end test loop

Goal: register an order, pay it, and watch the relayer deliver a **signed** webhook.

4.1 Start the dev webhook receiver (stands in for a merchant server)

```
cd backend
WEBHOOK_SECRET=whsec_test_1234567890 PORT=9000 npm run webhook-receiver
```

`src/dev/webhookReceiver.ts` verifies the `X-PayWithQuai-Signature` header against the secret and prints every payload on `POST /webhook`.

4.2 Onboard the demo merchant (admin API)

With `WEBHOOK_ALLOW_INSECURE_URLS=true` in the relayer `.env` (local dev), register the merchant payout address:

```
curl -X POST http://localhost:8080/v1/merchants \
-H "Authorization: Bearer $ADMIN_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "address": "0x00...your merchant payout wallet...",
  "name": "Demo Shop",
  "webhookUrl": "http://localhost:9000/webhook"
}'
```

`201` returns the `webhookSecret` **exactly once** — export it to the receiver (`WEBHOOK_SECRET=...`) and restart it. The payout address can be the deployer key used in Phase 1 (that's what `payDemo.js` assumes).

Other admin endpoints available for the test run:

```
curl -H "Authorization: Bearer $ADMIN_API_KEY" http://localhost:8080/v1/merchants
curl -X PATCH -H "Authorization: Bearer $ADMIN_API_KEY" -H "Content-Type: application/json" -d '{"active":true}' http://localhost:8080/v1/merchants/0x00...
curl -H "Authorization: Bearer $ADMIN_API_KEY" http://localhost:8080/v1/deliveries
```

4.3 Run the payment demo (ERC-20 path)

```
cd contracts
npm run demo:testnet # == hardhat run scripts/payDemo.js --network cyprus1
```

`scripts/payDemo.js` does the full loop:

1. Mints `25.00` mUSDQ to the payer (mock faucet).
2. Registers an order on-chain: `orderId = keccak256("ord_demo_<timestamp>")`, `token = mockStablecoin`, `amount = 25_000000` (6 decimals), `expiry = 0`.
3. Approves the proxy for the amount.
4. Calls `payOrder(merchant, orderId)` and prints the `PaymentReceived` event (merchant, orderId, payer, token, amount, timestamp).
5. Confirms `isSettled == true`.

4.4 Watch the relayer deliver the webhook

Within a few seconds (poll interval 5s, plus 12 confirmations on a fast testnet):

1. The **indexer** logs `payment confirmed – webhook queued` with the delivery id (`<txHash>: <logIndex>`), then `indexed block range`.
2. The **dispatcher** POSTs to `http://localhost:9000/webhook` with headers `X-PayWithQuai-Signature: t=<unix>,v1=<hmac>`, `X-PayWithQuai-Event: payment.confirmed`, `X-PayWithQuai-Delivery: <id>`.
3. The **receiver** prints `✅ payment.confirmed (<id>)` and the payload: `merchantId`, `merchant`, `orderId`, `payer`, `token`, `amount`, `feeBps`, `fee`, `net`, `txHash`, `blockNumber`, `timestamp` (amounts are decimal strings in the smallest unit).
4. The dispatcher logs `webhook delivered` — the delivery is now `delivered`.

4.5 Verify status endpoints

```
curl "http://localhost:8080/v1/orders/0x00...merchant.../0x...orderId..."
```

```
{
  "merchant": "0x00...",
  "orderId": "0x...",
  "token": "0x...",
  "amount": "25000000",
  "feeBps": 0,
  "expiry": "0",
  "settled": true,
  "webhook": { "status": "delivered", "attempts": 1 }
}
```

`/v1/deliveries` shows the delivery history (last 100). A failed delivery can be re-queued with `POST /v1/deliveries/:id/retry`.

4.6 Optional: native QUAJ path

`payDemo.js` covers ERC-20 only. To test the native path, register an order with `token = 0x0000000000000000000000000000000000000000000000000000000000000000` and have the payer call `payOrderNative(merchant, orderId)` with `msg.value` **exactly** equal to the registered amount (anything else reverts `IncorrectNativeValue`). The relayer treats `token == address(0)` as native QUAJ and reports it identically in the webhook.

4.7 Optional: unregistered-address catch-up

Pay an order with a merchant address that is **not** onboarded yet: the relayer records the delivery as `skipped` (`no merchant registered for payout address`). Onboard that address (§4.2) and the skipped delivery is automatically re-queued and delivered — a payment made before registration is never lost.

5. Verification checklist

#	Check	How
1	Contracts compile & unit tests pass	<code>npm test</code> in <code>contracts/</code>
2	Backend typecheck & unit tests pass	<code>npm run typecheck && npm test</code> in <code>backend/</code>
3	Deployment record written	<code>contracts/deployments/cyprus1.json</code> exists with a <code>payWithQuai</code> address
4	Relayer healthy	<code>GET /health</code> → <code>status: ok</code> , correct <code>contract</code> / <code>chainId</code> , cursor advancing
5	Merchant onboarded	<code>POST /v1/merchants</code> → 201 with <code>webhookSecret</code>
6	Order paid on-chain	<code>payDemo</code> prints <code>PaymentReceived</code> + <code>Order settled on-chain: true</code>

#	Check	How
7	Webhook signed & delivered	Receiver logs <code>✓ payment.confirmed</code> ; dispatcher logs <code>webhook delivered</code>
8	Status API reflects payment	<code>GET /v1/orders/...</code> → <code>settled: true</code> , <code>webhook.status: delivered</code>
9	Signature rejects tampering	Receiver returns <code>400 invalid signature</code> for a modified secret/body

6. Troubleshooting

Symptom	Likely cause	Fix
Deploy fails with insufficient funds	Deployer not funded in Cyprus-1	Fund the <code>0x00...</code> deployer from the Orchard faucet
<code>PAYWITHQUAI_ADDRESS ... is not a valid Quai zone address</code>	Address not in the zone matching the RPC	Use the proxy address from <code>deployments/<network>.json</code> on the same zone
Relayer finds no old payments	<code>START_BLOCK</code> empty → cursor seeded at <code>head - CONFIRMATIONS</code>	Set <code>START_BLOCK</code> to the proxy deployment block and restart (fresh <code>DATABASE_PATH</code>)
Webhook never delivered	Webhook URL blocked by SSRF guard / not https	Set <code>WEBHOOK_ALLOW_INSECURE_URLS=true</code> only in local dev; use public https in prod
Onboarding <code>400</code>	Address fails checksum or URL invalid/unsafe	Checksummed address; public https URL
Onboarding <code>409</code>	Address already registered	Use <code>PATCH /v1/merchants/:address</code> ; the secret is never rotated
Delivery stuck <code>pending</code> with growing <code>attempts</code>	Endpoint not 2xx in 10s, or 3xx redirect	Return 2xx fast; never redirect; check <code>/v1/deliveries</code> <code>lastError</code>
Delivery <code>failed</code>	<code>WEBHOOK_MAX_ATTEMPTS</code> exhausted	<code>POST /v1/deliveries/:id/retry</code> after fixing the endpoint
<code>429 too many requests</code>	Public orders route rate limit (60/min/IP)	Slow down polling; add cache
<code>unhandledRejection – exiting</code>	Async path failed (process exits by design)	Read the log line; process manager restarts cleanly
<code>OrderAlreadySettled</code> at demo	The demo order id collided	Fresh timestamp in <code>ord_demo_<ts></code> makes this unlikely; unique order ids are the rule

Symptom	Likely cause	Fix
MockStablecoin missing	Ran on mainnet (<code>CHAIN_ID=9</code>)	Deploy skips the mock on mainnet; use <code>STABLECOIN_ADDR</code>

7. Production deployment notes

- **Mainnet:** `RPC_URL=https://rpc.quai.network`, `CHAIN_ID=9`. `MockStablecoin` is **not** deployed — set `STABLECOIN_ADDR` to the real stablecoin to allowlist it. Never deploy the open-mint faucet to mainnet.
- **Governance:** set `MULTISIG_ADDR` (Gnosis Safe) so upgrades go through the timelock; complete the `acceptOwnership()` hand-off (§2.4).
- **Indexing:** set `START_BLOCK` to the proxy's deployment block on the very first relayer boot, so no payment history is missed.
- **Webhooks:** production webhook URLs must be https and publicly reachable; keep `WEBHOOK_ALLOW_INSECURE_URLS=false` (the default).
- **Secrets:** `ADMIN_API_KEY` ≥ 16 random chars; protect `DATABASE_PATH` (plaintext webhook secrets inside — created `0600`).
- **Process supervision:** the relayer exits intentionally on unhandled errors — run it under systemd/PM2/containers with auto-restart.
- **Frontend:** `frontend/` is currently the default Next.js scaffold (no checkout UI is implemented yet); merchants bring their own checkout calling `registerOrder` / `payOrder` (see the Merchant Integration Guide in this folder).